

Sadržaj

Uvod	1
1. Problem navigacije unutar skladišnog okruženja	2
1.1. Opis problema	2
1.2. Implementacija okoline sustava.....	2
1.3. Primjena podržanog učenja	3
2. Implementacija simulacijskog sustava	4
2.1. Arhitektura aplikacije	4
2.2. Implementacija skladišnog okruženja	7
2.2.1. Model skladišta.....	7
2.2.2. Upravljanje simulacijom.....	8
3. Rješavanje problema primjenom Q-učenja	9
3.1. Algoritam Q-učenja	9
3.2. Proces učenja algoritma Q-učenja	10
3.2.1. Bellmanova jednačba.....	11
3.3. Implementacija algoritma	13
3.4. Prednosti i ograničenja Q-učenja.....	16
4. Eksperimenti i analiza rezultata.....	17
4.1. Početna konfiguracija sustava	18
4.2. Utjecaj veličine mape sustava.....	19
4.3. Utjecaj hiperparametara na proces učenja	22
4.3.1. Utjecaj parametra ϵ	22
4.3.2. Utjecaj stope učenja α	24
4.3.3. Utjecaj faktora diskontiranja γ	26
Zaključak	29
Literatura	30
Sažetak.....	31
Summary.....	32

Uvod

Automatizacija poslovanja sve je veća i dostupnija, tako primjerice i navigacija robota unutar njegovog prostora za rad. Postoje razni algoritmi i procedure za pretraživanje prostora stanja, poput heurističkog i metaheurističkog pretraživanja. U ovom radu napravljen je pokušaj primjene tipa strojnog učenja, zvan podržano učenje. Specifičnije, fokus algoritma strojnog učenja u ovom radu je Q-učenje (engl. *Q-learning*).

Q-učenje kao metoda rješavanja problema vrlo je zanimljiva jer se razlikuje od mnogih dostupnih metoda pretraživanja stanja. Q-učenje kroz pokušaje i pogreške (engl. *trial and error*) i interakciju s okruženjem, uči o okruženju u kojem se nalazi, te radi akcije na temelju samo stečenog znanja preko nagrada i kazni za odabranu akciju. U ovom radu primijeniti će se na pronalazak optimalnog puta od startne pozicije do paketa, te od paketa do krajnje točke, uz izbjegavanje prepreka i postupno učenje o radnom prostoru.

U prvom poglavlju rada opisan je problem navigacije unutar skladišnog okruženja, načini modeliranja okoline te motivacija za primjenu podržanog učenja. U drugom poglavlju prikazana je arhitektura implementiranog simulacijskog sustava i uloga pojedinih modula. Treće poglavlje opisuje teoriju i implementaciju algoritma Q-učenja, uključujući Bellmanovu jednadžbu i strategiju odabira akcija. U četvrtom poglavlju provedeni su eksperimenti i analiza rezultata, s naglaskom na utjecaj hiperparametara i veličine prostora stanja na učinkovitost algoritma, brzinu konvergencije, stabilnost procesa učenja i broj potrebnih epizoda do postizanja optimalne politike.

Cilj ovog rada je vidjeti koliko je dobro rješenje algoritam Q-učenja za problem navigacije unutar zadanog nepoznatog prostora.

1. Problem navigacije unutar skladišnog okruženja

1.1. Opis problema

Problem koji se nastoji riješiti je pronalazak optimalne putanje od početne lokacije preko nasumično odabrane pozicije paketa do krajnje točke. Početna lokacija bi trebala simulirati mjesto u kojem robot boravi, a krajnja lokacija mjesto za ostavljanje pokupljenog paketa. Nasumično postavljen paket je odabran iz razloga da se pokaže nezavisnost lokacije paketa i okruženja od algoritma učenja agenta.

U usporedbi s tradicionalnim algoritmima za navigaciju, naš agent na početku ne zna ništa o svom okruženju. Nema nikakvog plana kretanja. Tijekom procesa učenja, on mora sam odlučiti koje radnje su dobre, a koje su loše. To znači da mora naučiti što radi dobro, a što loše. Ovo je problem odlučivanja, gdje agent uči kroz interakciju sa okolinom.

1.2. Implementacija okoline sustava

U sklopu rada, napravili smo aplikaciju koristeći programski jezik Python 3.13 i biblioteku Pygame. Aplikacija prikazuje sučelje, animacije i kretanje agenta u odabranom prostoru.

Biblioteka `Pygame` je skup Python modula namijenjenih razvoju igara i ostalih multimedijских aplikacija. Nadogradnja je na postojeću izvrsnu SDL biblioteku. Omogućuje upravljanje grafičkim sučeljem, animacijama i korisničkim unosom u stvarnom vremenu [1].

`Pygame` biblioteka odabrana je zbog mogućnosti jednostavne integracije s programskim jezikom Python, te potpunoj mogućnosti kontrole nad prikazanom simulacijom. Svi vizualni elementi su zapravo slike koje se uz pomoć biblioteke prikazuju na ekranu korisnika. Slike su napravljene koristeći alat Microsoft Designer.

Prostor na kojem agent uči modeliran je kao dvodimenzionalno polje preko matrice koristeći `numpy` biblioteku, te svako polje predstavlja jednu poziciju unutar skladišta. Detalji vizualne implementacije i korištenje biblioteka opisani su kasnije u radu.

1.3. Primjena podržanog učenja

Podržano učenje (engl. *reinforcement learning*) je područje strojnog učenja u kojem agent uči donositi odluke nakon interakcije s okolinom. Naspram nadziranog učenja, agentu nisu unaprijed poznati primjeri ispravnog ponašanja, odnosno skup podataka nad kojim bi se on mogao izvježbati. Umjesto toga, tijekom izvođenja akcija prima povratne informacije u obliku nagrada i kazni, te na temelju njih postupno razvija strategiju ponašanja [2].

U promatranom problemu, agent predstavlja autonomnog robota, dok okolinu čini skladišni prostor sa preprekama, lokacijom paketa, polazištem i krajnjim ciljem. Kroz pozitivne nagrade, potiče se poželjno ponašanje poput uspješnog preuzimanja paketa ili dolaska na cilj sa paketom. Kroz kazne, sprječavamo neželjeno ponašanje agenta poput sudara sa preprekama ili kretanje po putu do paketa/cilja čija duljina nije optimalna.

Problem optimalnog pronalaska putanje pripada skupini epizodičkih problema. Svaka epizoda započinje postavljanjem agenta u isto početno stanje, te završava kada agent uspješno dostavi paket na određenu lokaciju ili kada se ispuni neki od uvjeta završetka simulacije, poput prevelikog broja koraka. Tijekom puno ponavljanja epizoda agent prikuplja znanje koje koristi za poboljšanje odluka u sljedećim epizodama [2].

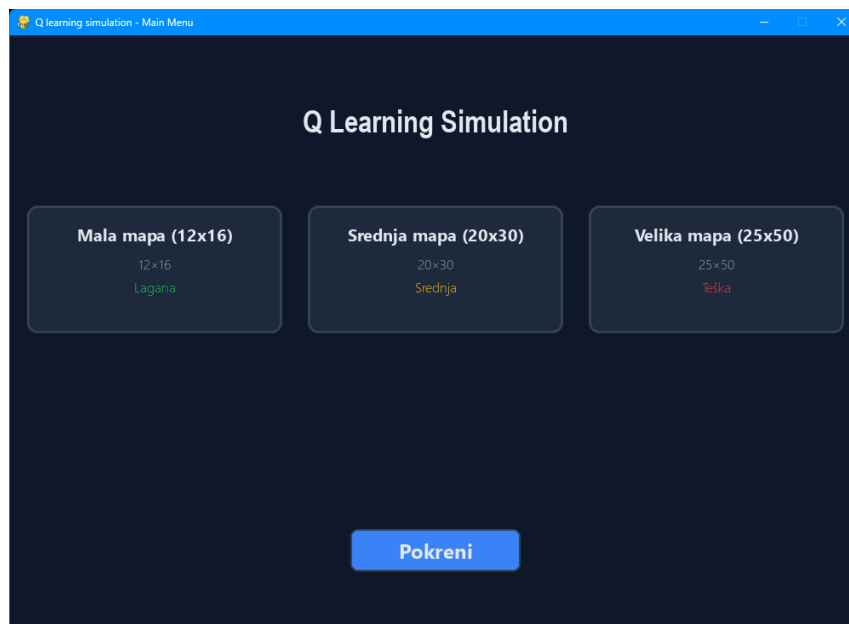
Ovakav pristup rješavanju problema pogodan je zbog mogućnosti primjene na mapama različitih i unaprijed nepoznatih konfiguracija, pri čemu agent nije ograničen unaprijed definiranim pravilima kretanja, već optimalnu strategiju razvija samostalno kroz iskustvo stečenim s interakcijom s okolinom.

2. Implementacija simulacijskog sustava

2.1. Arhitektura aplikacije

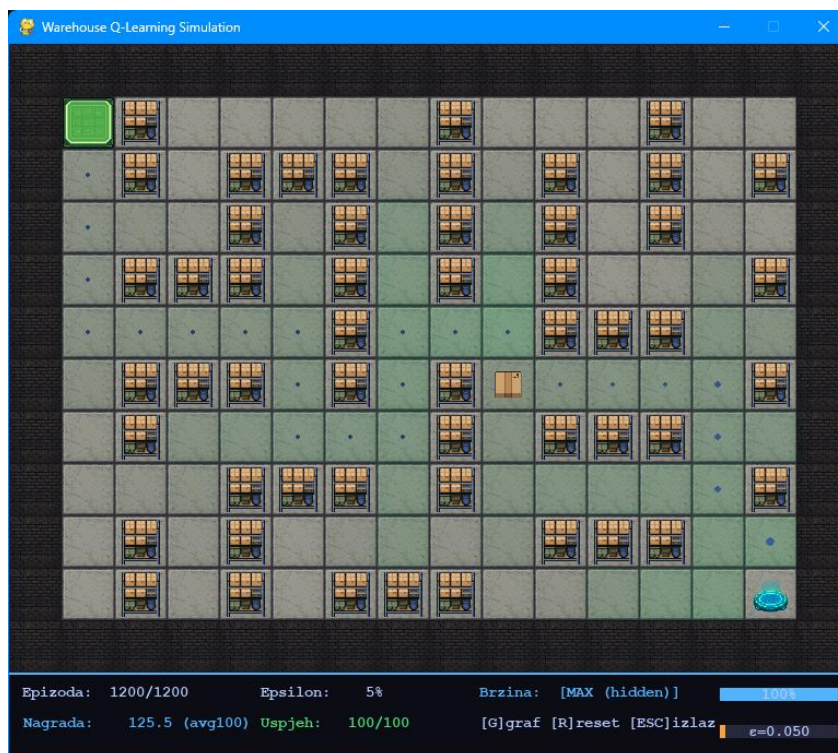
Aplikacija je organizirana u više međusobno povezanih modula. Takav pristup omogućuje jasnu podjelu funkcionalnosti, jednostavnije održavanje sustava i lakšu nadogradnju pojedinih komponenti.

Ulazna točka aplikacije nalazi se u datoteci `main.py` koja korisniku omogućuje odabir željene konfiguracije skladišta te pokretanje simulacije. Početni zaslon aplikacije prikazan je na slici (Sl. 2.1).



Sl. 2.1 Početni izbornik

Glavni radni dio sustava implementiran je u datoteci `simulation.py`, koja upravlja izvođenjem simulacije i procesom treniranja agenta. Ovaj modul povezuje okruženje, agenta i grafički prikaz te omogućuje izvođenje učenja na odabranoj skladišnoj mapi. Primjer izgleda simulacijskog okruženja tijekom izvođenja na mapi dimenzija 12×16 prikazan je na slici ispod (Sl. 2.2).



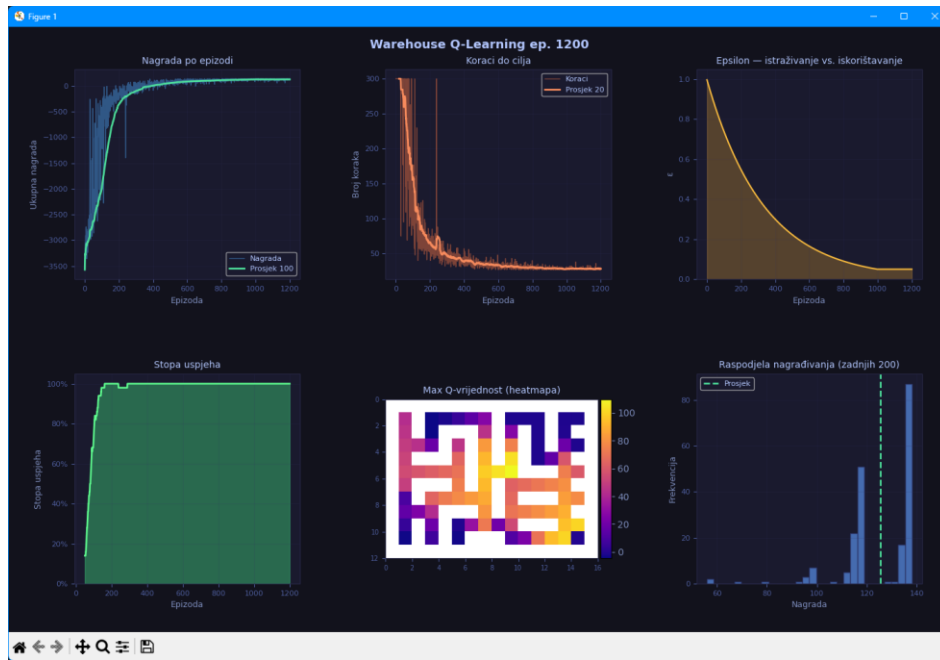
Sl. 2.2 Simulacija na mapi 12x16

Implementacija algoritma Q-učenja nalazi se u datoteci `agent.py`, gdje su definirani postupci odabira akcija, ažuriranja Q-vrijednosti i upravljanja strategijom istraživanja okruženja.

Za grafički prikaz simulacije koristi se `graphics.py` temeljen na biblioteci `Pygame`. Modul je zadužen za ubacivanje i kontroliranje slika generiranih alatom `Microsoft Designer` na određenu poziciju, te crtanje HUD-a.

Konfiguracijski parametri sustava definirani su u datoteci `config.py`, dok su konfiguracije skladišnih prostora pohranjene u datoteci `maps.json`.

Praćenje procesa učenja omogućeno je putem modula `graphs.py`. Prikazuju se grafovi: ostvarene nagrade po epizodi, broja potrebnih koraka do ostvarenja cilja, promjene vrijednosti parametra ϵ (epsilon), stope uspješnosti agenta te raspodjele ostvarenih nagrada. Generira se i toplinska mapa (engl. *heatmap*) koja prikazuje maksimalne Q-vrijednosti za pojedina polja okoline, čime je moguće vizualno analizirati koje dijelove prostora agent smatra najvrjednijima tijekom procesa učenja. Primjer grafičkog prikaza pokazatelja treniranja prikazan je na Sl. 2.3.



Sl. 2.3 Prikaz grafova nakon treniranja modela 12x16

Rezultati eksperimenata pohranjuju se pomoću modula `experiment_logger.py` u direktorij `results` u formatu CSV (engl. Comma Separated Values), čime je omogućena naknadna analiza i usporedba različitih konfiguracija treniranja.

2.2. Implementacija skladišnog okruženja

2.2.1. Model skladišta

Skladišno okruženje u simulaciji modelirano je kao dvodimenzionalna diskretna mreža polja. Svako polje predstavlja jednu lokaciju unutar skladišta te može biti slobodno ili zauzeto preprekom. Takav način modeliranja omogućuje jednostavno definiranje prostora za kretanje agenta i prikladan je za primjenu algoritama podržanog učenja u problemima navigacije. U radu su korištene tri konfiguracije skladišnih prostora različitih dimenzija: 12×16 , 20×30 i 25×50 polja. Korištenjem mreža polja različitih veličina moguće je analizirati utjecaj složenosti okruženja na proces učenja te usporediti ponašanje agenta u manjim i većim skladišnim prostorima.

Konfiguracije skladišta pohranjene su unutar `maps.json` datoteke, iz koje se gradi simulacija ovisno o odabranoj mapi na početnom izborniku. Datoteka sadrži:

- naziv mape
- dimenzije skladišta (broj redaka i stupaca)
- početnu poziciju agenta
- ciljnu poziciju za dostavu paketa
- popis koordinata prepreka
- više unaprijed definiranih konfiguracija skladišta različitih dimenzija i složenosti

Na temelju učitane konfiguracije generira se matrica skladišta gdje vrijednost 0 predstavlja slobodno polje, dok vrijednost 1 predstavlja prepreku ili zid skladišta.

Opisano okruženje modelirano je kao Markovljev proces odlučivanja (engl. *Markov Decision Process*) gdje buduće stanje sustava ovisi isključivo o trenutnom stanju i odabranoj akciji, što je temeljno Markovljevo svojstvo i preduvjet za primjenu algoritma Q-učenja [6].

2.2.2. Upravljanje simulacijom

Logika izvođenja simulacije implementirana je unutar datoteke `simulation.py`. Simulacija se izvršava kroz glavnu petlju koja kontinuirano:

- obrađuje korisničke događaje
- izvršava korake treniranja
- ažurira stanje agenta
- računa nagrade
- ponovno iscrtava elemente okruženja

Tijekom izvođenja simulacije korisnik može upravljati načinom rada aplikacije pomoću tipkovnice. Implementirane su funkcionalnosti za:

- pauziranje i nastavak simulacije
- povećavanje i smanjivanje brzine izvođenja
- uključivanje i isključivanje grafova
- te kontroliranja brzine treniranja

Sustav podržava više načina izvođenja simulacije:

- standardni vizualni prikaz,
- ubrzano izvođenje simulacije,
- i maksimalno brzo treniranje bez prikaza animacija.

Takav pristup omogućava detaljno praćenje ponašanja agenta tijekom simulacije, ali i brzo izvođenje velikog broja epizoda potrebnih za treniranje algoritma Q-učenja.

Osim same simulacije, tijekom izvođenja mogu se kontinuirano pratiti podaci o učenju agenta poput:

- ukupne nagrade,
- broja koraka,
- uspješnosti dostave,
- i trenutne vrijednosti parametra epsilon.

3. Rješavanje problema primjenom Q-učenja

3.1. Algoritam Q-učenja

Q-učenje (engl. *Q-learning*) je model-free algoritam podržanog učenja koji omogućava agentu učenje optimalne strategije kretanja kroz iskustvo i interakciju s okruženjem. Agent tijekom procesa učenja istražuje različite akcije te na temelju dobivenih nagrada postupno procjenjuje koje akcije u određenim stanjima daju najbolje rezultate [4].

Temelj algoritma predstavlja Q-tablica u kojoj se za svaki par stanja i akcije pohranjuje procijenjena vrijednost korisnosti određene akcije. Tijekom velikog broja epizoda agent ažurira Q-vrijednosti i postupno razvija optimalnu strategiju kretanja [4]. Naziv Q-learning dolazi od Q-vrijednosti (engl. *Quality values*).

Svaki redak tablice predstavlja jedno stanje sustava, dok stupci predstavljaju moguće akcije agenta. U simulaciji agent može napraviti četiri osnovne akcije kretanja:

- gore
- dolje
- lijevo
- desno

Stanje sustava definirano je pozicijom agenta i informacijom posjeduje li agent paket, čime se omogućava razlikovanje situacija prije i nakon preuzimanja paketa. Prema ovim informacijama možemo izračunati koliki je broj stanja potreban za Q tablicu prema izrazu (3.1.). Broj stanja jednak je broju polja na mapi pomnoženom s dvije moguće vrijednosti koja označava posjeduje li agent paket ili ne. To je bitno jer želimo da agent pokupi kutiju prije nego što dođe do krajnjeg cilja.

$$N_{STATES} = GRID_{HEIGHT} * GRID_{WIDTH} * 2 \quad (3.1.)$$

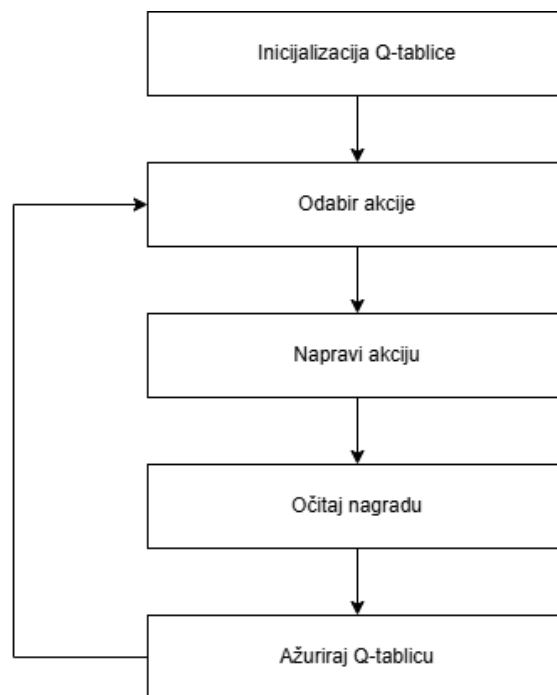
Veličina Q-tablice računa se prema izrazu (3.2.), te je jednako broju mogućih stanja pomnoženim sa brojem akcija koje agent može napraviti, a to je 4.

$$N_{TABLE} = N_{STATES} * 4 \quad (3.2.)$$

Znači da za mapu male veličine 12x16 postoji 384 stanja i tablica veličine 1536 elemenata, srednje veličine 20x30 postoji 1200 stanja i tablica veličine 4800, te za veliku mapu 25x50 2500 stanja i tablica veličine 10000.

3.2. Proces učenja algoritma Q-učenja

Algoritam Q-učenja temelji se na iterativnom procesu učenja u kojem agent postupno poboljšava svoje odluke kroz interakciju s okruženjem. Dijagram procesa prikazan je na Sl. 3.1. Tijekom svake epizode agent promatra trenutno stanje okoline, odabire akciju, prima nagradu te na temelju dobivenih informacija ažurira svoje znanje. Ponavljanjem velikog broja epizoda agent postupno uči koje akcije u pojedinim stanjima dovode do najveće ukupne nagrade [4].



Sl. 3.1 Dijagram toka algoritma Q-učenja

Na početku svake epizode, agent se nalazi u početnom stanju koje opisuje njegovu trenutnu situaciju unutar prostora (skladišnog). Nakon određivanja trenutnog stanja agent odabire jednu od dostupnih akcija. Odluka se donosi na temelju politike odlučivanja koje koristi informacije pohranjene u Q-tablici. Tijekom treniranja koristi se ϵ -pohlepna (engl. *ϵ -greedy*) strategija odabira akcija. S vjerojatnošću ϵ agent odabire nasumičnu akciju kako bi istraživao prostor stanja, dok u suprotnom odabire akciju s najvećom Q-vrijednošću. Kako bi se postupno smanjivala potreba za istraživanjem i povećala iskorištenost naučenog znanja,

koristi se mehanizam ϵ -decay prikazan u isječku koda Kod 3.2. To se implementira tako da se ϵ vrijednost svakom novom epizodom smanjuje, odnosno što se dulje model trenira, smanjuje se šansa za odabir nasumično odabrane akcije i odabire se naučena akcija iz Q-tablice, prikazano u isječku koda Kod 3.1.

Nakon odabira akcije, agent je izvršava i dolazi do prijelaza u novo stanje. Okruženje agentu daje povratnu informaciju u obliku nagrade ili kazne koja opisuje kvalitetu donesene odluke. Dobiveno iskustvo se zatim koristi za ažuriranje Q-tablice. Agent prilagođava Q-vrijednost za odgovarajući par stanje-akcija na temelju primljene nagrade/kazne i procjene budućih nagrada koje može ostvariti iz novog stanja. Tako Q-vrijednosti postupno nakon sve više i više epizoda postaju preciznija procjena kvalitete odabira svake akcije. Postupnim prikupljanjem znanja, razvija strategiju koja omogućuje pronalazak što veće ukupne nagrade (odnosno optimalne putanje).

```
def choose_action(self, state):
    if random.random() < self.epsilon:
        return random.randint(0, self.n_actions - 1)
    return int(np.argmax(self.Q[state]))
```

Kod 3.1 Odabir akcije u ovisnosti ϵ parametra

```
self.epsilon = max(EPS_END, self.epsilon * EPS_DECAY)
```

Kod 3.2 Implementacija ϵ -decay

3.2.1. Bellmanova jednadžba

Ažuriranje Q-vrijednosti temelji se na Bellmanovoj jednadžbi (3.3.) [5]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)) \quad (3.3.)$$

Pri čemu je:

- $Q(s, a)$ – trenutnu procjenu korisnosti izvođenja akcije a u stanju s
- α – stopa učenja (engl. *learning rate*)
- r – dobivena nagrada/kazna, odnosno dobit nakon izvršene akcije
- γ – diskontni faktor (engl. *discount factor*)
- s' – novo stanje
- a' – nova akcija
- $\max Q(s', a')$ - maksimalna Q-vrijednost u idućem stanju s' za neku akciju a'

U implementaciji se ažuriranje Q-tablice izvršava pomoću Kod 3.3:

```
best_next = 0 if done else np.max(self.Q[s2])
self.Q[s, a] += ALPHA * (r + GAMMA * best_next - self.Q[s, a])
```

Kod 3.3 Ažuriranje Q-tablice

Na taj način agent nakon svake akcije prilagođava procjenu kvalitete pojedinih odluka na temelju prethodnog iskustva i očekivanih budućih nagrada.

Koristeći Bellmanovu jednadžbu (3.3.), odluke se ne donose isključivo na temelju trenutačne nagrade, već uzima u obzir i procjenu budućih nagrada koje agent može ostvariti iz novog stanja. Time agent razvija dugoročnu strategiju ponašanja umjesto donošenja samo trenutačno najbolje odluke.

U izrazu (3.3.) veličina $Q(s, a)$ predstavlja trenutačnu procjenu korisnosti izvođenja akcije a u stanju s . Nagrada r opisuje neposrednu korist ili kaznu koju je agent ostvario izvršavanjem akcije. Parametar α predstavlja stopu učenja, te određuje koliki će utjecaj nova informacija imati na postojeću Q-vrijednost. Veće vrijednosti omogućuju bržu prilagodbu, dok manje vrijednosti rezultiraju sporijim, ali stabilnijim učenjem [5].

Parametar γ predstavlja diskontni faktor kojim se određuje važnost budućih nagrada. Veće vrijednosti daju veću važnost dugoročnim koristima, dok manje vrijednosti favoriziraju neposredne nagrade[5].

Izraz $\max Q(s', a')$ predstavlja najveću procijenjenu vrijednost među svim mogućim akcijama u novom stanju s' .

Razlika između izraza $r + \gamma \max Q(s', a')$ i trenutne vrijednosti $Q(s, a)$ naziva se pogreška temporalne razlike (engl. *Temporal Difference Error*) [5]. Ona predstavlja odstupanje između prethodne procjene i nove informacije dobivene iz iskustva. Ako je ostvareni rezultat bolji od očekivanog, Q-vrijednost će se povećati, dok će se u suprotnom smanjiti. Ponavljanjem velikog broja epizoda Q-vrijednosti postupno konvergiraju prema stvarnim očekivanim nagradama.

3.3. Implementacija algoritma

Glavni dio implementacije algoritma napisan je u modulu `agent.py` koji sadrži klasu `QLearningAgent`. U ovoj klasi se nalazi logika algoritma Q-učenja, uključujući inicijalizaciju Q-tablice, odabira akcija korištenjem ϵ -pohlepne strategije, ažuriranje Q-vrijednosti primjenom Bellmanove jednadžbe i izvođenje procesa treniranja kroz pojedine epizode. U njoj se isto tako spremaju statistički podatci potrebni za kasniju analizu učenja.

Prilikom inicijalizacije klase, Q-tablica postavlja sve vrijednosti na nule koristeći `np.zeros`, što znači da agent na početku ne pretpostavlja nikakvu korisnost niti jedne akcije. Parametar ϵ se inicijalizira na vrijednost 1.0, što odgovara potpunom istraživanju okruženja pri početnoj epizodi.

Stanje sustava preslikava se na jedinstveni cijeli broj, odnosno ID, na temelju pozicije agenta i informaciji je li pokupio paket. Ovaj dio implementacije je nužan jer agent mora razlikovati situacije u kojima ima paket ili nema, a nalazi se na istoj poziciji jer su to funkcionalno potpuno različita stanja.

Proces učenja izvodi se unutar metode `run_episode()`. Metodi se predaju informacije o okolini kroz parametre metode, poput mreže skladišta, poziciji paketa, dostupne akcije i funkcije za preslikavanje stanja u ID i provjeru valjanosti pozicije, te popis nagrada/kazni. Glavna petlja prikazana je u isječku koda Kod 3.4.

```
for step in range(MAX_STEPS):
    a = self.choose_action(s)
    dr, dc = actions[a]
    nr, nc = r + dr, c + dc

    if not valid(nr, nc):
        reward = rewards['obstacle']
        ns = s
        nr, nc = r, c
    else:
        if (nr, nc) == pickup_pos and not has_item:
            has_item = True
            reward = rewards['pickup']
        elif (nr, nc) == goal and has_item:
            reward = rewards['goal']
            ns = state_id(nr, nc, has_item)
```

```

        self.update(s, a, reward, ns, True)
        success = True
        break
    elif (nr, nc) == goal and not has_item:
        reward = rewards['obstacle']
        ns = s
        nr, nc = r, c
    else:
        reward = rewards['step']

    ns = state_id(nr, nc, has_item)
    if ns in visited:
        reward += rewards['revisit']
    visited.add(ns)

self.update(s, a, reward, ns, False)
s, r, c = ns, nr, nc

```

Kod 3.4 Glavna petlja epizode učenja

U svakom koraku epizode agent odabire akciju pozivom metode `choose_action()` koja implementira ϵ -pohlepnu strategiju opisanu u prethodnom poglavlju. Na temelju odabrane akcije izračunava se nova pozicija agenta. Ako nova pozicija nije valjana, odnosno predstavlja zid ili izlazi izvan granica mape, agent ostaje na trenutnoj poziciji i prima kaznu. U suprotnom se provjerava što se nalazi na novoj poziciji te se dodjeljuje odgovarajuća nagrada ili kazna prema sljedećoj logici:

- agent je stigao do paketa kojeg još nije pokupio → prima nagradu i postavlja se zastavica posjedovanja paketa
- agent je stigao do ciljne pozicije s paketom → prima nagradu, epizoda završava uspješno
- agent je stigao do ciljne pozicije bez paketa → prima kaznu i ostaje na trenutnoj poziciji
- agent je napravio normalan korak kretanja → prima malu kaznu koja potiče pronalazak kraćih (optimalnih) putanja
- agent je napravio korak na već posjećeno polje → prima kaznu kako bi se izbjeglo kruženje agenta

Na kraju svakog koraka poziva se metoda `update()` koja ažurira Q-tablicu primjenom Bellmanove jednadžbe opisane i izvedene u prethodnom poglavlju.

Nakon završetka treniranja dostupna je metoda `get_best_path()` koja se koristi nakon treniranja, odnosno nakon odrađenih svih epizoda. Poziva se kako bi se prikazala naučenost agenta na odabranoj mapi. Primjer za mapu 20x30 se nalazi na Sl. 3.2. Ona koristi isključivo pohlepnu politiku, odnosno u svakom koraku odabire akcije s najvećom Q-vrijednosti, tako dobivajući najbolji naučeni put.



Sl. 3.2 Prikaz najbolje putanje na mapi 20x30

Na kraju svake epizode pohranjuju se statistički podatci o ostvarenoj nagradi, broju koraka i uspješnosti dolaska do cilja. Ti podatci koriste se za praćenje napretka učenja putem grafova i za pohranu rezultata eksperimenata. Osim toga, nakon svih epizoda vizualizira se toplinska mapa maksimalnih Q-vrijednosti prikazana na Sl. 3.2 zelenom bojom. Što je polje intenzivnije zeleno, to agent smatra tu poziciju vrijednijom na putu prema cilju. Na početku treniranja vrijednosti su niske i ravnomjerno raspoređene jer agent još nije naučio koje pozicije vode do cilja, dok se nakon dovoljnog broja epizoda jasno ocrtava naučena optimalna putanja.

3.4. Prednosti i ograničenja Q-učenja

Jedna od glavnih prednosti algoritma Q-učenja je njegova jednostavnost implementacije i sposobnost učenja bez prethodnog poznavanja modela okoline. Također omogućuje relativno jednostavno eksperimentiranje s različitim sustavima nagrađivanja i hiperparametrima, što je korisno pri analizi ponašanja agenta u različitim uvjetima. Agent uči donositi bolje odluke, bez vanjskog nadzora, i s vremenom postaje sve bolji.

S druge strane, algoritam ima i određena ograničenja. Kod većih prostora stanja veličina Q-tablice brzo raste, što povećava memorijske zahtjeve i usporava proces učenja. Također, agent često zahtijeva velik broj epizoda kako bi naučio stabilnu strategiju ponašanja. Zbog navedenog ograničenja na standardni algoritam Q-učenja, koriste se metode dubokog podržanog učenja poput Deep Q-Network (DQN) algoritma, koje umjesto tablice koristi duboke neuronske mreže za aproksimaciju Q vrijednosti, i time rješava problem prevelikog broja stanja.

Još jedno ograničenje algoritma jest da u praktičnim primjenama nije garantiran pronalazak globalno optimalnog rješenja. Iako je teorijski dokazano da Q-učenje konvergira prema optimalnoj politici uz dovoljno istraživanje prostora stanja i velik broj epizoda, u stvarnim uvjetima uspješnost učenja ovisi o odabranim hiperparametrima, veličini prostora stanja te dostupnom broju epizoda za treniranje. Primjerice kod jako složenih problema sa puno stanja, nije nam garantirano da će u zadanom broju epizoda agent proći dovoljan broj stanja dovoljno puta da razvije optimalnu politiku, ali bi trebao pronaći dovoljno dobru.

4. Eksperimenti i analiza rezultata

Cilj provedenih eksperimenata je ispitati uspješnost algoritma Q-učenja nad danim problemom pronalaska optimalnog puta unutar skladišnog okruženja. Glavni zadatak agenta tijekom treniranja je naučiti kako u najkraćem mogućem broju koraka doći do nasumično postavljenog paketa, te ga sigurno dostaviti na ciljnu lokaciju, maksimizirajući nagrade izbjegavanjem prepreka i minimiziranjem broja koraka.

Kako bi se dobio detaljan i objektivan uvid u ponašanje algoritma, tijekom svake epizode učenja kontinuirano su prikupljeni podatci. Uspješnost modela vrednovana je kroz praćenje kretanja prosječne i ukupne nagrade, konvergencije prema manjem broju koraka do cilja i ukupne stope uspješnosti agenta od početka do paketa pa do cilja.

U nastavku poglavlja rezultati su podijeljeni u dvije cjeline. Prvo će se analizirati ponašanje agenta i stabilnost učenja kroz različite veličine i složenosti mreža polja. U drugom dijelu će se ispitivati utjecaji hiperparametara na jednoj mapi, odnosno njihovo djelovanje na brzinu konvergencije i kvalitetu konačne naučene politike kretanja.

Grafovi prikazani u nastavku su nacrtani pomoću modula `plot_experiments.py` i biblioteke `matplotlib` koristeći podatke koje program sprema u obliku CSV datoteke na kraju učenja svake konfiguracije.

4.1. Početna konfiguracija sustava

U svim eksperimentima korištena je osnovna konfiguracija algoritma Q-učenja definirana u datoteci config.py. Ako nije drugačije navedeno, korištene su sljedeće vrijednosti hiperparametara u Tablica 4.1.

Parametar	Vrijednost
Broj epizoda	1200
Maksimalan broj koraka	300
Stopa učenja (α)	0.7
Diskontni faktor (γ)	0.95
Početni epsilon	1.0
Završni epsilon	0.05
Epsilon decay	0.997

Tablica 4.1 Tablica vrijednosti hiperparametara korištenih u eksperimentima

Sustav nagrada definiran je u Tablica 4.2:

Događaj	Nagrada/kazna
Dostava paketa	+100
Preuzimanje paketa	+50
Sudar s preprekom	-20
Jedan korak	-0.5
Povratak u posjećeno stanje	-1

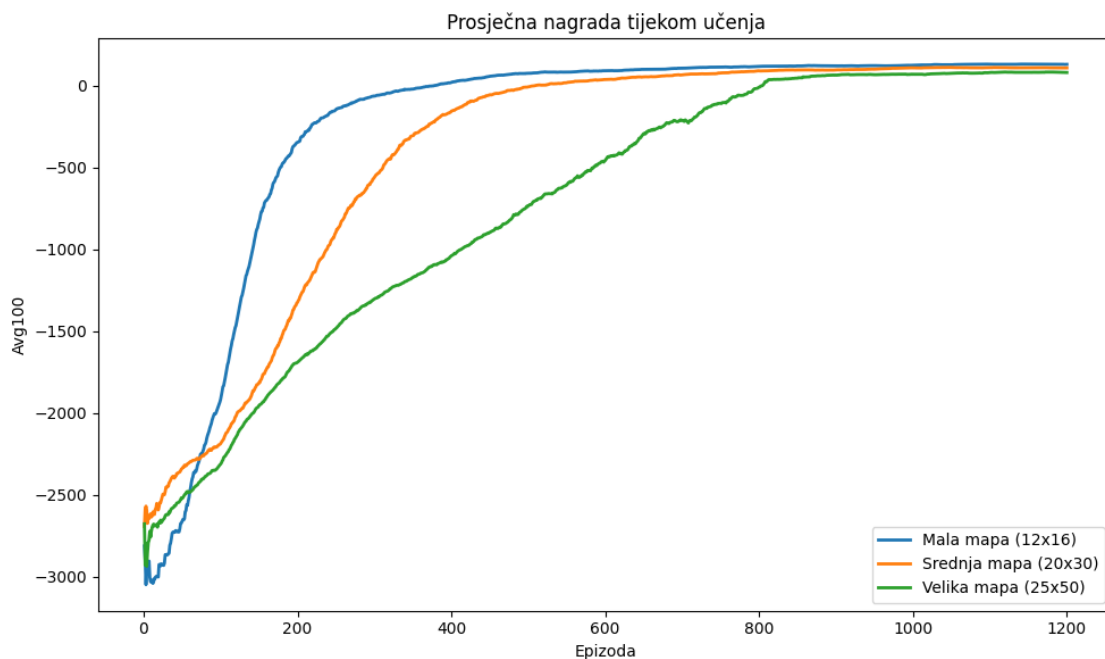
Tablica 4.2 Tablica vrijednosti politike nagrađivanja korištene u eksperimentima

Kao referentna konfiguracija okruženja korištena je mapa dimenzija 20x30 polja, osim u eksperimentima u kojima se posebno ispituje utjecaj veličine skladišta. Također, radi kvalitetnije analize, paket neće biti nasumično raspoređen, već će biti fiksiran na jednu lokaciju.

4.2. Utjecaj veličine mape sustava

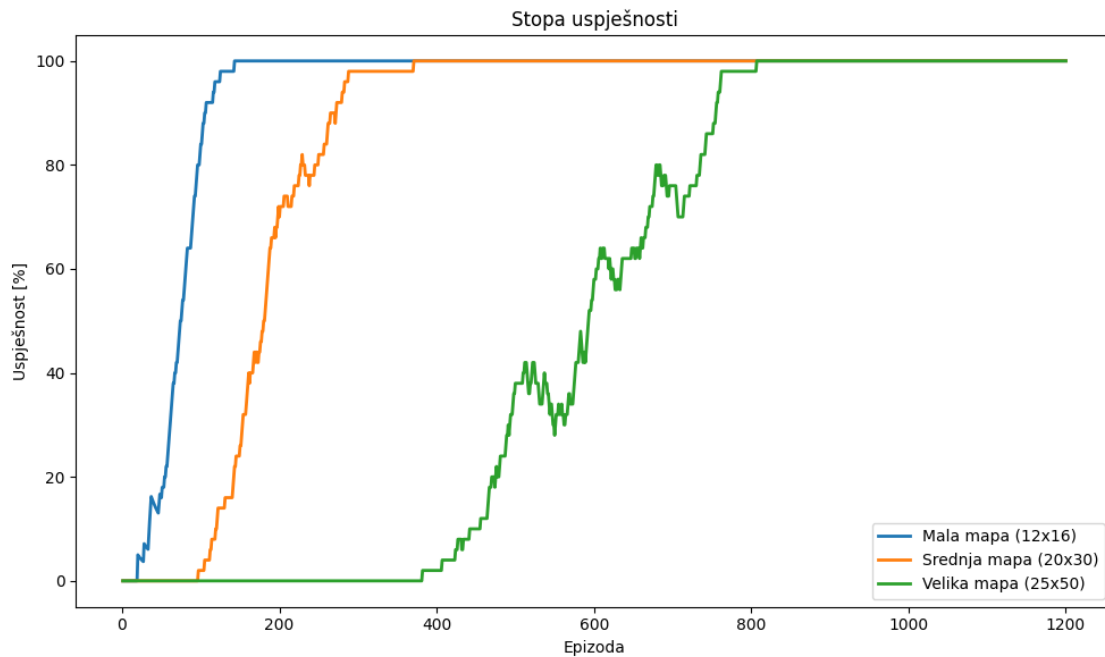
Kako bismo bolje razumjeli ograničenja algoritma Q-učenja, važno je analizirati kako se agent ponaša pri skaliranju okoline. Povećanjem dimenzija skladišta ne raste samo udaljenost koju robot mora prijeći, već se drastično povećava broj mogućih stanja u Q-tablici. Zato su provedeni eksperimenti na tri različite konfiguracije skladišta: 12x16, 20x30 i 25x50 polja. Sve ostale postavke sustava i hiperparametri ostali su jednaki kako bi se izolirao isključivo utjecaj veličine prostora.

Gledajući graf prosječne nagrade (Sl. 4.1), jasno se uočava razlika u brzini kojom agent svladava okolinu. Na najmanjoj mapi, agent vrlo brzo pronalazi paket i ciljnu lokaciju, zbog čega prosječna nagrada strmo raste već u ranim epizodama. S druge strane, mapa 25x50 predstavlja znatno veći izazov. Budući da agent na početku uči isključivo kroz nasumično istraživanje, u velikom skladištu puno dulje luta i samim time prikuplja više kazni za svaki odrađeni korak. Zbog toga krivulja nagrade za najveću mapu raste puno sporije, a u konačnici postiže i nešto niže maksimalne vrijednosti jer optimalni put prirodno zahtijeva više koraka (a time i više malih negativnih nagrada).



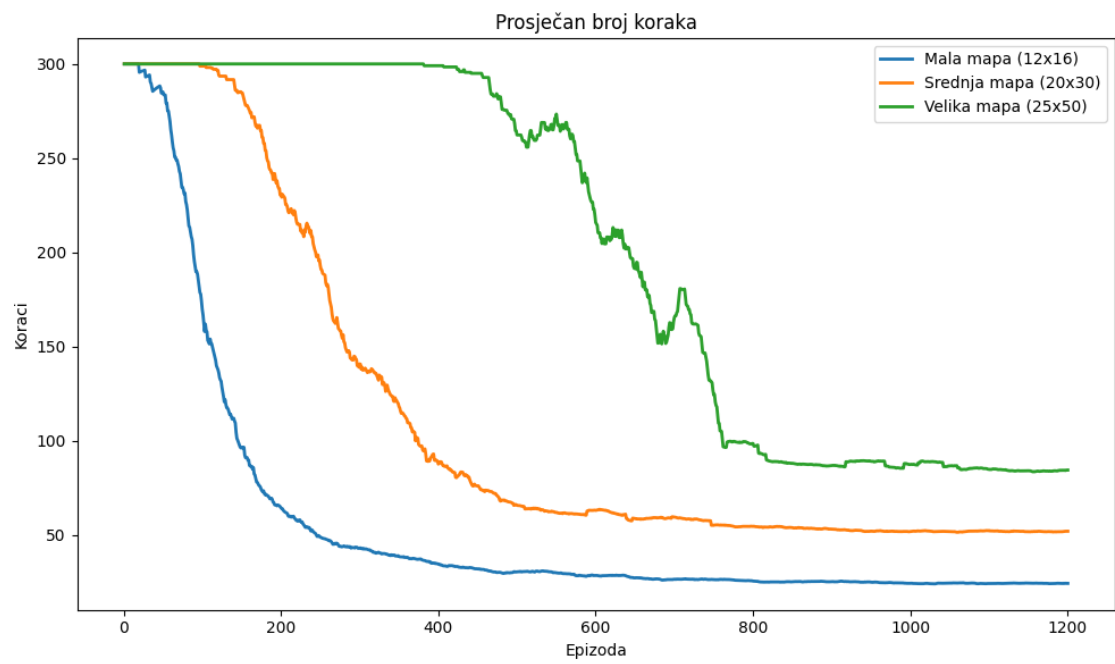
Sl. 4.1 Graf prosječne nagrade na mapama različitih veličina

Slično ponašanje agenta može se primijetiti i na grafu stope uspješnosti (Sl. 4.2). Iako se u početnim epizodama na najvećoj mapi agent često gubi ili mu istekne maksimalan broj koraka prije pronalaska cilja, pozitivno je to što algoritam s vremenom ipak uspijeva pronaći rješenje. Dok na maloj mapi agent gotovo odmah doseže visoku razinu uspješnosti, kod veće mape potrebno mu je oko 800 epizoda više da eliminiira loše odluke, poput besciljnog kruženja ili zapinjanja uz prepreke. Ipak, pred kraj treniranja sve tri konfiguracije postižu visoku i stabilnu uspješnost.



Sl. 4.2 Graf stope uspješnosti na mapama različitih veličina

Konačna potvrda da je agent usvojio dobru politiku kretanja vidljiva je iz prosječnog broja koraka potrebnih za završetak epizode (Sl. 4.3). U početku učenja, broj koraka je visok za sve mape jer agent uči koristeći veliki ϵ parametar. Međutim, kako se Q-vrijednosti sve preciznije ažuriraju i ϵ pada, broj koraka u svim scenarijima oštro pada i na kraju se stabilizira. Ta točka stabilizacije označava trenutak kada agent prestaje s pretjeranim istraživanjem i počinje primjenjivati naučenu, najkraću putanju do cilja.



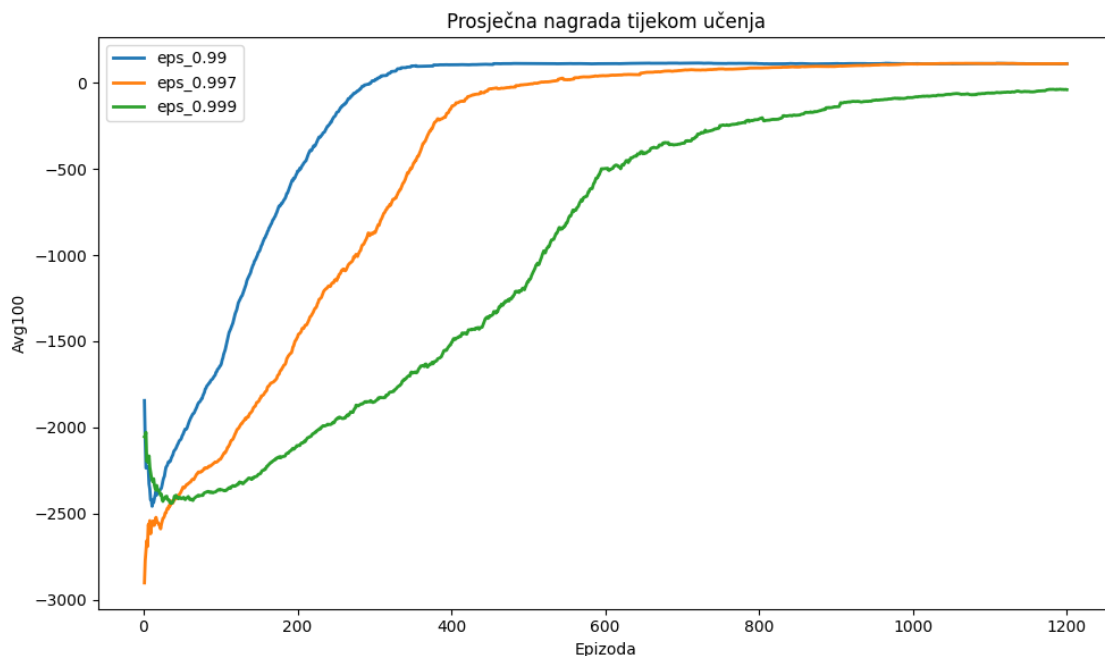
Sl. 4.3 Graf broja koraka agenta na mapama različitih veličina

4.3. Utjecaj hiperparametara na proces učenja

4.3.1. Utjecaj parametra ϵ

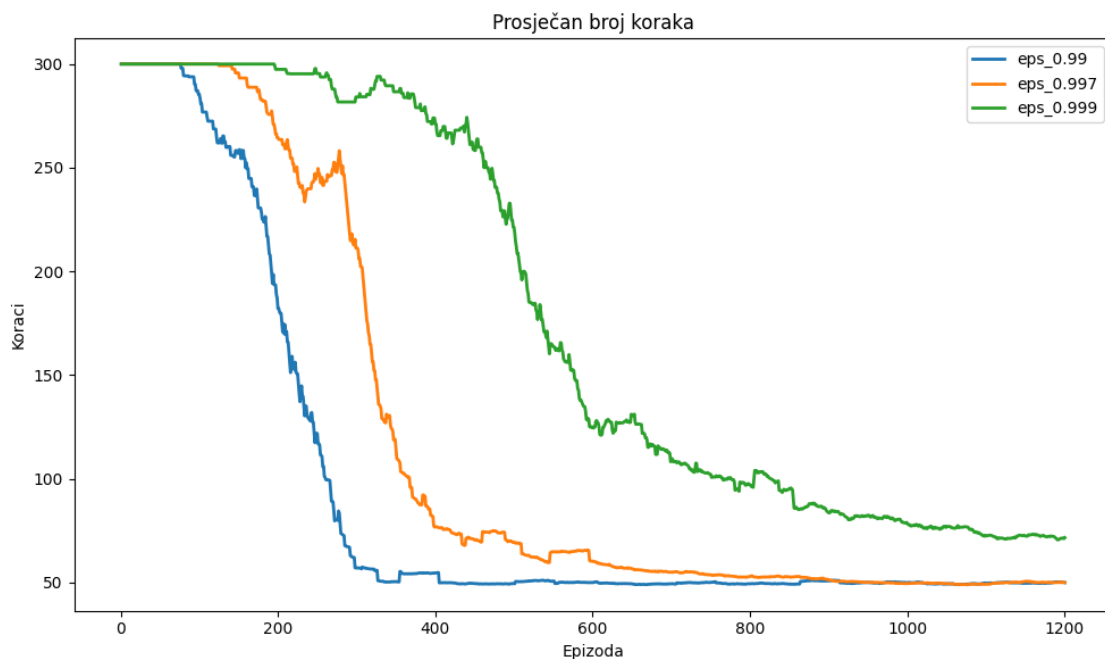
Parametar ϵ određuje vjerojatnost odabira nasumične akcije tijekom procesa učenja te predstavlja kompromis između istraživanja novih stanja (engl. *exploration*) i iskorištavanja već naučenih informacija (engl. *exploitation*). U ovom eksperimentu analiziran je utjecaj brzine opadanja parametra ϵ na performanse algoritma. Korištene su tri vrijednosti parametra EPS_DECAY: 0.990, 0.997 i 0.999 (ekvivalentno vjerojatnostima 1%, 0.3% i 0.1% povećanja šansi odabira akcije prema Q-tablici umjesto nasumično u svakoj sljedećoj epizodi), dok su svi ostali parametri i konfiguracija okruženja ostali nepromijenjeni.

Prateći kretanje prosječne nagrade (Sl. 4.4), vrlo se jasno očitavaju različiti "karakteristi" učenja ovisno o zadanoj vrijednosti. Kada smo postavili brzo opadanje (0.990), agent je vrlo rano prestao s nasumičnim istraživanjem i počeo se oslanjati isključivo na ono malo iskustva koje je do tada prikupio. Iako to dovodi do brze početne stabilizacije nagrade, takav pristup nosi velik rizik jer postoji mogućnost da agent nauči neoptimalnu putanju, te misli da je ona optimalna.

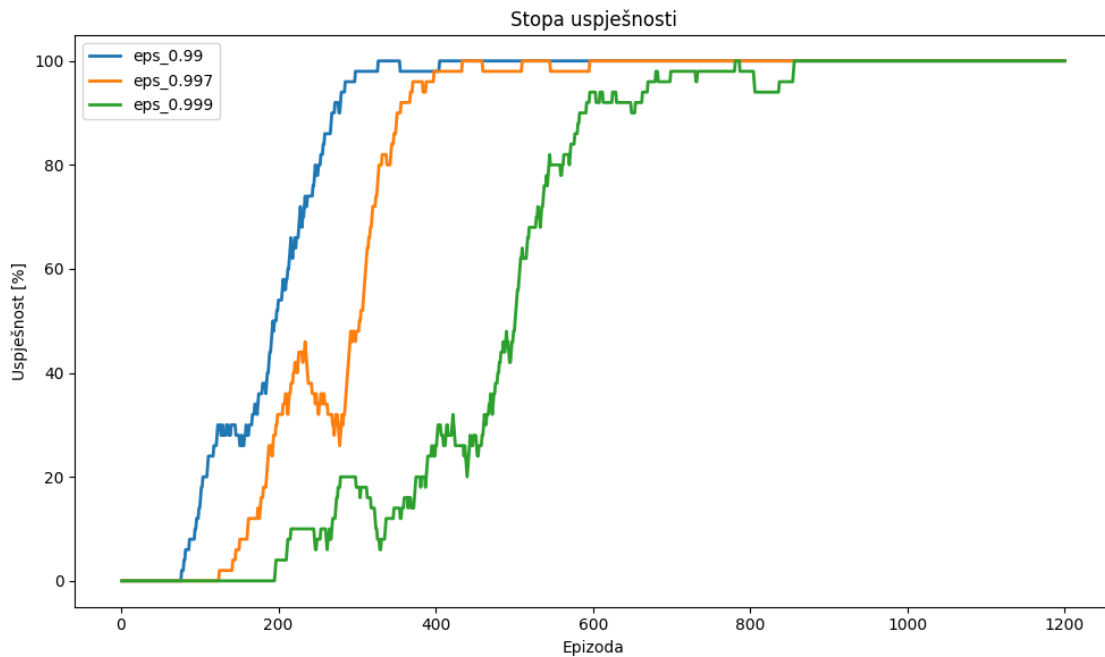


Sl. 4.4 Graf prosječne nagrade za različite vrijednosti ϵ -decay

Ova dinamika je zanimljiva kada detaljnije promotrimo graf prosječnog broja koraka po epizodi (Sl. 4.5). Kod brzog opadanja ϵ , broj koraka se stabilizira vrlo rano, ali često na višoj razini jer je agent prestao istraživati prije nego što je našao najkraći put. S druge strane, kod vrijednosti 0.999, na grafu se može primijetiti da agent čak i pri samom kraju treniranja (oko 1200. epizode) radi više koraka od očekivanog. Na prvi pogled, ovo bi moglo sugerirati da algoritam nije uspio pronaći najbolje rješenje, međutim to nije istina. Agent je pronašao optimalnu putanju u svojoj Q-tablici. Razlog za prividno veći broj koraka je zbog izrazito sporog opadanja vrijednost parametra ϵ , tako u 1200. epizodi još uvijek relativno visoka (30% šanse da odabere nasumično akciju). Ovo odlično ilustrira ključnu razliku u podržanom učenju između onoga što agent radi tijekom treniranja (gdje i dalje namjerno griješi radi istraživanja) i onoga što je zapravo naučio. Ista stvar se može primijetiti i na grafu stope uspješnosti (Sl. 4.6), gdje samo kod 0.999 ϵ -decay vrijednosti uspješnost nije konvergirala na 100%.



Sl. 4.5 Graf prosječnog broja koraka za različite vrijednosti ϵ -decay



Sl. 4.6 Graf stope uspješnosti za različite vrijednosti ϵ -decay

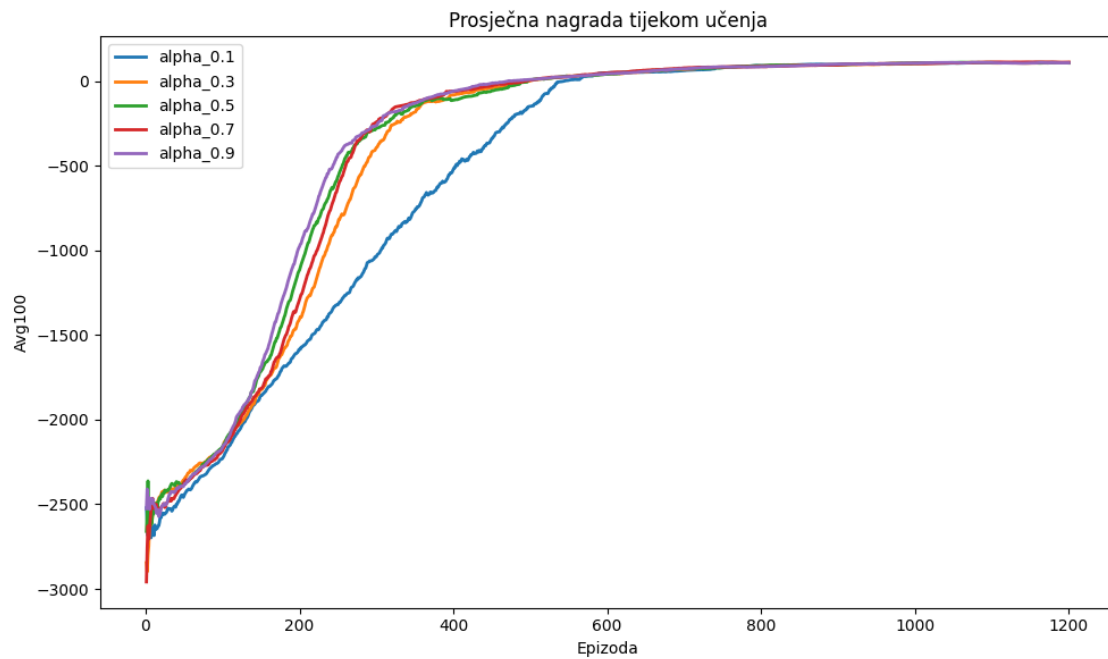
4.3.2. Utjecaj stope učenja α

Nakon analize parametra ϵ , idući korak je ispitati kako stopa učenja α utječe na proces učenja. Cilj je utvrditi može li veća vrijednost parametra α , koja omogućuje brže ažuriranje Q-vrijednosti, ubrzati konvergenciju algoritma. U tu svrhu testiran je širi raspon vrijednosti parametra α : 0.1, 0.3, 0.5, 0.7 i 0.9.

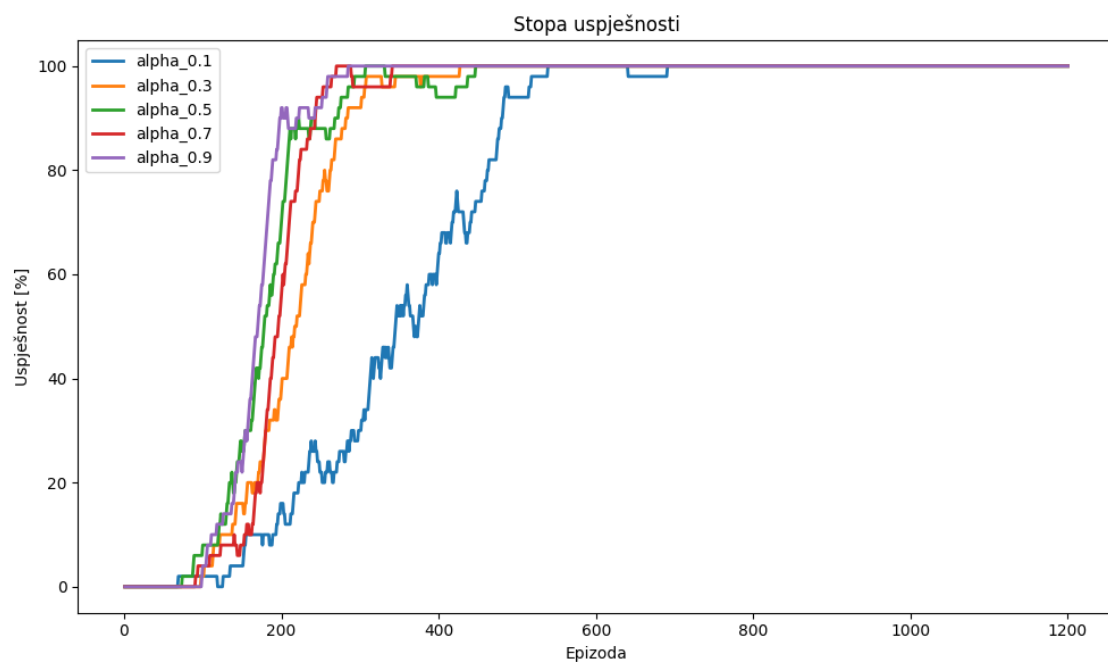
Na grafovima prosječne nagrade (Sl. 4.7), stope uspješnosti (Sl. 4.8) i prosječnog broja koraka (Sl. 4.9), jedino se konfiguracija s najnižom vrijednosti ($\alpha = 0.1$) primjetno razlikovala od ostalih. Model s $\alpha = 0.1$ je također uspio pronaći optimalan put i naučiti zadatak, ali mu je trebalo osjetno više epizoda da postigne konvergenciju. S druge strane, sve ostale vrijednosti (od 0.3 do 0.9) pokazale su vrlo slične i stabilne performanse, brzo pronalazeći najkraći put.

Ovakvo ponašanje može se objasniti načinom na koji je postavljen sustav nagrade u simulaciji. Budući da su nagrade za preuzimanje paketa (+50) i dolazak na cilj (+100) jako velike u usporedbi s kaznom za korak (-0.5), pogreška temporalne razlike (TD) pri prvom uspjehu je ogromna. Zbog velike razlike, vrijednosti od 0.3 na više su sasvim dovoljne da se proces učenja ubrza. Vrijednost od 0.1 se pokazala presporom za tako brzo širenje. Zbog

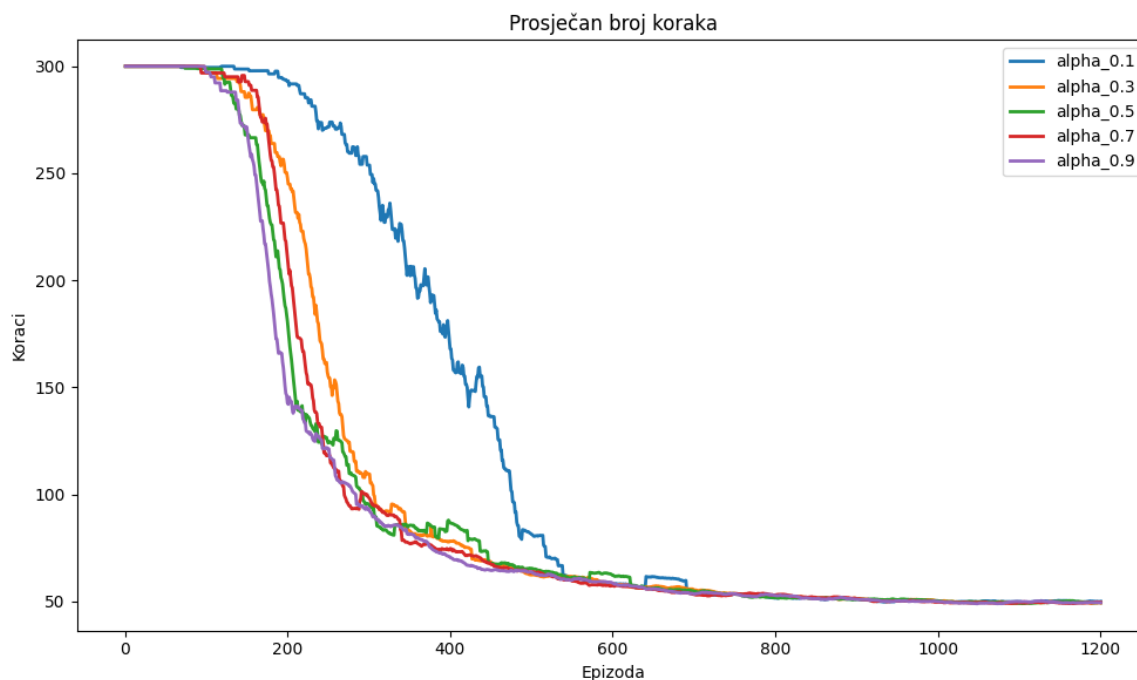
toga što model dobro reagira na više vrijednosti i pritom ne gubi stabilnost, primijenjena je zadana vrijednost stope učenja u kodu na $\alpha = 0.7$, jer se pokazala kao najbolji izbor.



Sl. 4.7 Graf prosječne nagrade za različite vrijednosti α



Sl. 4.8 Graf stope uspješnosti za različite vrijednosti α



Sl. 4.9 Graf broja koraka za različite vrijednosti α

4.3.3. Utjecaj faktora diskontiranja γ

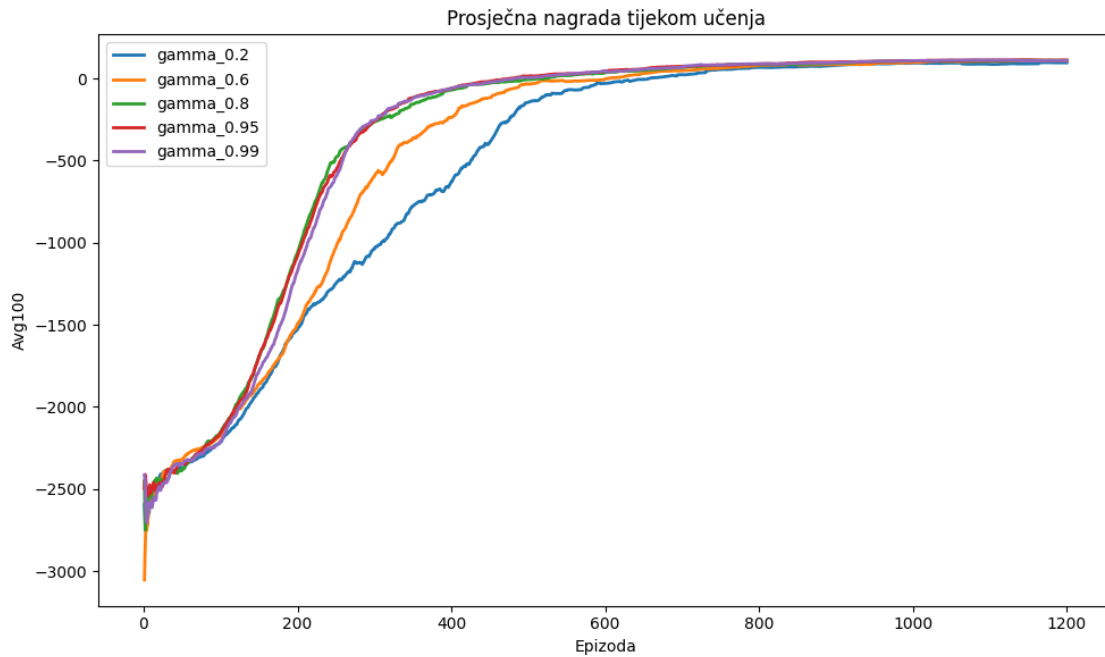
Za razliku od stope učenja, pokazalo se da faktor diskontiranja γ znatno više utječe na rezultate učenja. Kako bi se ispitalo koliko usmjerenost agenta na trenutačne nagrade utječe na pronalazak puta, testirane su vrijednosti $\gamma = 0.2, 0.6, 0.8, 0.95$ i 0.99 .

Na dobivenim grafovima jasno se vidi razlika među postavkama. Najbolje i najstabilnije učenje postignuto je za $\gamma = 0.95$. Međutim, već pri smanjenju parametra na $\gamma = 0.6$ uočavaju se prvi problemi. Na grafu prosječnog broja koraka (Sl. 4.12) vidi se da pronađena putanja više nije najkraća moguća. Također, na grafu prosječne nagrade (Sl. 4.10) i stope uspješnosti (Sl. 4.11) primjećuje se sporije učenje. Krivulje u kasnijim epizodama nisu ravne, već blago osciliraju, što znači da agent povremeno donosi lošije odluke i ne bira uvijek najbolji mogući put.

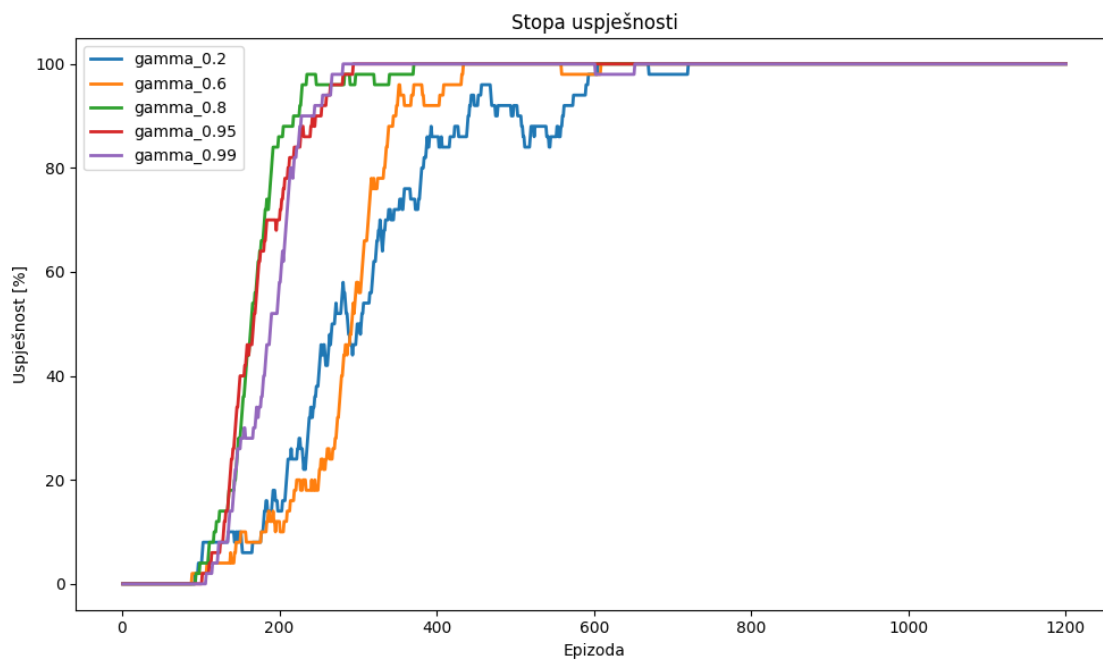
Značajno lošiji rezultat se dogodio pri najnižoj testiranoj vrijednosti, $\gamma = 0.2$. Agent s ovom postavkom uzima u obzir gotovo isključivo neposredne nagrade, te u konačnici ne pronalazi optimalan put. Na grafu broja koraka (Sl. 4.12) vidljive su velike oscilacije tijekom cijelog procesa učenja jer agent većinom luta. Budući da ovako mala vrijednost faktora diskontiranja prebrzo smanjuje vrijednost udaljene nagrade, agentu koji se nalazi na startnoj poziciji velika nagrada od +100 ne znači gotovo ništa. On u obzir uzima samo idućih nekoliko koraka koji

mu donose kaznu od -0.5, zbog čega gubi motivaciju za dolazak do cilja. Iz tog razloga, nagrada i uspješnost rastu sporije i nastavljaju oscilirati čak i u kasnijim epizodama.

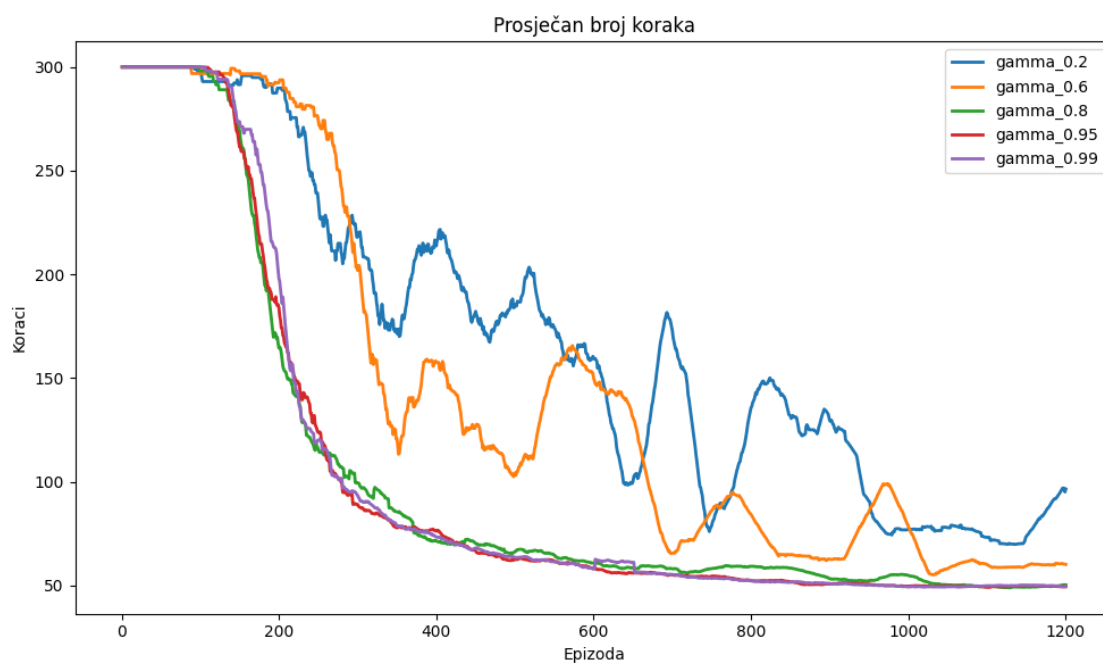
Ovaj eksperiment jasno potvrđuje da je za problem navigacije u prostoru dugoročno planiranje vrlo bitno za proces učenja algoritma. Za konačnu verziju modela odabrana je vrijednost $\gamma = 0.95$. Ta vrijednost omogućuje agentu da planira dovoljno koraka unaprijed kako bi uspješno pronašao cilj, dok istovremeno proces učenja ostaje maksimalno stabilan.



Sl. 4.10 Graf prosječne nagrade za različite vrijednosti γ



Sl. 4.11 Graf stope uspješnosti za različite vrijednosti γ



Sl. 4.12 Graf broja koraka za različite vrijednosti γ

Zaključak

U ovom radu primijenjen je algoritam Q-učenja na problemu navigacije autonomnog agenta (robota) u simuliranom pojednostavljenom skladišnom okruženju. Odabrani programski jezik je Python, a za simulaciju je korištena popularna biblioteka `Pygame` koja se uglavnom koristi za razvoj igrica u Python-u.

Problem navigacije pojednostavljen je problem pronalaska optimalne putanje od početne pozicije, do pozicije paketa pa do krajnjeg cilja. Svodi se na navigaciju u prostoru, odnosno zaobilazanje prepreka.

Provođenjem eksperimenata promjenom vrijednosti hiperparametara i složenosti okoline, analiziran je rad algoritma Q-učenja i kako se mijenja nad različitim konfiguracijama. Iako Q-učenje nije namijenjeno za pretraživanje prostora stanja, uspješno je riješilo dani problem. Od ispitanih hiperparametara, faktor diskontiranja γ pokazao se najutjecajnijim na kvalitetu naučene politike, dok je stopa opadanja parametra ϵ bila ključna za ravnotežu između istraživanja i iskorištavanja naučenog znanja.

Glavni nedostatak korištenog pristupa algoritma Q-učenja je mogućnost proširenja. Veličina Q-tablice raste s dimenzijama okoline i dodatnim uvjetima koji određuju stanje, što za jako velike ili nediskretne prostore stanja postaje nepraktično. Mogući nastavak ovog rada bila bi primjena dubokog Q-učenja, spomenuto u radu kao DQN (engl. *Deep Q Network*), koje neuronskom mrežom aproksimira Q-vrijednosti i time prevladava ograničenja tabličnog pristupa, otvarajući mogućnost primjene na znatno složenijim okruženjima.

Literatura

- [1] *Pygame community, Pygame Documentation*, (2026.). Poveznica: <https://www.pygame.org/wiki/about>; pristupljeno 2. lipnja 2026.
- [2] Dalbelo Bašić B., Čupić M., Šnajder J. *Uvod u umjetnu inteligenciju – Podržano učenje*. Poveznica: <https://www.fer.unizg.hr/download/repository/predavanje.pdf>
- [3] Čupić M., *Umjetna inteligencija: podržano učenje*, 1. izdanje, 2020
- [4] *Q-Learning in Reinforcement Learning*, GeeksforGeeks, Poveznica: <https://www.geeksforgeeks.org/machine-learning/q-learning-in-python/>; pristupljeno 1. lipnja 2026.
- [5] *An Introduction to Q-Learning*, datacamp. Poveznica: <https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial>; pristupljeno 2. lipnja 2026.
- [6] *Markov Decision Process*, GeeksForGeeks, Poveznica: <https://www.geeksforgeeks.org/machine-learning/markov-decision-process/>; pristupljeno 2. lipnja 2026.
- [7] *Matplotlib, Matplotlib 3.10.9 documentation*, Poveznica: <https://matplotlib.org/stable/>; pristupljeno 2. lipnja 2026.

Sažetak

Primjena podržanog učenja za upravljanje autonomnim agentom u 2D
simuliranom skladišnom okruženju

U ovome radu primjenjuje se algoritam podržanog učenja, odnosno Q-učenje za upravljanje agentom u 2D simulacijskom programu. Zadatak agenta je pronalazak optimalne putanje od početne pozicije do nasumično postavljenog paketa te njegova dostava na odredišnu lokaciju, uz izbjegavanje prepreka. Opisana je implementacija algoritma koristeći Python, te biblioteku Pygame za gradnju simulacijskog okruženja. Objašnjena je teorija Q-učenja, te provedena analiza i utjecaj hiperparametara i veličina prostora stanja na funkcionalnost algoritma.

Ključne riječi: podržano učenje, strojno učenje, Q-učenje, Pygame, autonomni agent

Summary

Application of Reinforcement Learning for Autonomous Agent Control in a 2D Simulated Warehouse Environment

This paper applies a reinforcement learning algorithm, specifically Q-learning, to control an autonomous agent within a 2D simulation environment. The agent's task is to find the optimal path from its starting position to a randomly placed package and then deliver it to a designated destination while avoiding obstacles. The paper describes the implementation of the algorithm using Python programming language and the Pygame library for building the simulation environment. The theoretical background of Q-learning is explained, followed by an analysis of how different hyperparameters and state space sizes affect the algorithm's overall performance and functionality.

Keywords: reinforcement learning, machine learning, Q-learning, Pygame, autonomous agent.